Ruijie Portal Voucher API Documentation

Base URL

```
https://portal-as.ruijienetworks.com
```

Authentication Flow

1. Get Session ID

Method: `GET`

URL: Portal session URL (contains gateway parameters)

Description: Redirects to obtain a `sessionId` parameter from the final URL.

Query Parameters:

Parameter	Type	Description
gw_id	string	Gateway ID
gw_address	string	Gateway IP address
gw_port	string	Gateway port
mac	string	Client MAC address
ip	string	Client IP address

Response: Redirect URL containing `sessionId` parameter.

---

2. Get Captcha Image

Method: `GET`

URL: `/api/auth/captcha/image`

Query Parameters:

Parameter	Type	Description

sessionId	string	Session ID from step 1
_t	string	Current timestamp

Response: Image binary (PNG/JPEG captcha image)

Status Codes:

Code	Meaning
200	Success - returns captcha image
400	Invalid session ID
500	Server error

---

3. Verify Captcha

Method: `POST`

URL: `/api/auth/captcha/verify`

Content-Type: `application/json`

Request Body:

JSON

```
{
  "sessionId": "string",
  "authCode": "string"
}
```

Field	Type	Description
sessionId	string	Current session ID
authCode	string	Captcha text (user input)

Response:

JSON

```
{
  "success": true
}
```

Status Codes:

Code	Meaning
200	Verification result returned
400	Missing parameters
500	Server error

Response Fields:

Field	Type	Description
success	boolean	`true` if captcha is correct, `false` otherwise

---

4. Validate Voucher Code

Method: `POST`

URL: `/api/auth/voucher/?lang=en_US`

Content-Type: `application/json`

Request Body:

JSON

```
{
  "accessCode": "string",
  "sessionId": "string",
  "apiVersion": 1,
  "authCode": "string"
}
```

Field	Type	Description

accessCode	string	Voucher code to validate (6-8 digits/characters)
sessionId	string	Current session ID
apiVersion	integer	API version, always `1`
authCode	string	Verified captcha text

Success Response (valid code):

JSON

```
{
  "logonUrl": "https://...",
  "message": "Authentication successful"
}
```

Failed Response (invalid code):

JSON

```
{
  "error": "Invalid access code",
  "code": "AUTH_FAILED"
}
```

Rate Limited Response:

JSON

```
{
  "message": "request limited",
  "code": "RATE_LIMITED"
}
```

Status Codes:

Code	Meaning
200	Request processed (check response body for result)
400	Bad request / missing parameters

429	Rate limited
500	Server error

Response Indicators:

Response Contains	Meaning
`logonUrl`	Voucher code is valid and authenticated
`STA`	Code is limited/already used
`request limited`	Rate limited, retry later

---

5. Get Balance

Method: `GET`

URL: `/api/auth/balance/getBalance/{sessionId}`

Path Parameters:

Parameter	Type	Description
sessionId	string	Session ID after successful authentication

Response:

JSON

```
{
  "totalMinutes": 1440,
  "remainingMinutes": 1200,
  "success": true
}
```

Response Fields:

Field	Type	Description
totalMinutes	integer	Total plan duration in minutes

remainingMinutes	integer	Remaining time in minutes
remainingSeconds	integer	Remaining time in seconds (alternative)
success	boolean	Request success status

Status Codes:

Code	Meaning
200	Balance info returned
404	Session not found
500	Server error

Voucher Code Format

- **6-digit numeric:** `000000` to `999999`
- **7-digit numeric:** `0000000` to `9999999`
- **8-digit numeric:** Random 8-digit numbers
- **Alphanumeric:** 6-character lowercase letters or mixed

Rate Limiting

- Requests may be rate-limited with `"request limited"` response
- Recommended: Implement retry logic with backoff
- Maximum concurrent requests: Configurable per deployment

Security Notes

- Session IDs expire after inactivity
- Captcha verification required before voucher validation
- MAC address randomization supported for session rotation